

Assignment 1

-Submitted by Ayush Patel(BT2024054)

Assignment 0

Setup:

- **CPU Model & Cores:** Apple M4, 10 Cores (4 Performance, 6 Efficiency)
- **RAM:** 16 GB
- **HDD/SSD Capacity:** 245.11 GB
- **OS Version:** macOS 26.5.2
- **PostgreSQL Engine Version:** 16.15 (Homebrew)

Database:

Name: Join Order Benchmark (2013 Snapshot)

Reference to schematext.sql

The below is info in order -> Table , PK , cardinality(obtained by run count(*)):

Here from schema i see that PK is id only for all 21 tables , the table name and rows are mentioned below:

I used a script because i knew sql but the union command is assisted by gemini so i can get correct ans : the script is->

```
SELECT 'aka_name' AS table_name, count(*) AS cardinality FROM aka_name
```

```
UNION ALL
```

```
SELECT 'aka_title', count(*) FROM aka_title
```

```
UNION ALL
```

```
SELECT 'cast_info', count(*) FROM cast_info
```

```
UNION ALL
```

```
SELECT 'char_name', count(*) FROM char_name
```

```
UNION ALL
```

```
SELECT 'comp_cast_type', count(*) FROM comp_cast_type
```

```
UNION ALL
```

```
SELECT 'company_name', count(*) FROM company_name
```

```
UNION ALL
```

```

SELECT 'company_type', count(*) FROM company_type
UNION ALL
SELECT 'complete_cast', count(*) FROM complete_cast
UNION ALL
SELECT 'info_type', count(*) FROM info_type
UNION ALL
SELECT 'keyword', count(*) FROM keyword
UNION ALL
SELECT 'kind_type', count(*) FROM kind_type
UNION ALL
SELECT 'link_type', count(*) FROM link_type
UNION ALL
SELECT 'movie_companies', count(*) FROM movie_companies
UNION ALL
SELECT 'movie_info', count(*) FROM movie_info
UNION ALL
SELECT 'movie_info_idx', count(*) FROM movie_info_idx
UNION ALL
SELECT 'movie_keyword', count(*) FROM movie_keyword
UNION ALL
SELECT 'movie_link', count(*) FROM movie_link
UNION ALL
SELECT 'name', count(*) FROM name
UNION ALL
SELECT 'person_info', count(*) FROM person_info
UNION ALL
SELECT 'role_type', count(*) FROM role_type
UNION ALL
SELECT 'title', count(*) FROM title;

```

Output is->

table_name	cardinality
company_type	4
comp_cast_type	4
kind_type	7
role_type	12
link_type	18
info_type	113
movie_link	29997
complete_cast	135086

```

keyword      |    134170
company_name |    234997
aka_title    |    361472
aka_name     |    901343
movie_info_idx |   1380035
movie_companies |   2609129
title        |   2528312
char_name    |    3140339
movie_keyword |   4523930
person_info  |    2963664
movie_info   |   14835720
name         |    4167491
cast_info    |   36244344
(21 rows)

```

Assignment 1

Q1. Find out the histograms for columns “id” and “title” of the “title” table.

- 1. What is the corresponding SQL?**
- 2. What kind of histogram is it?**
- 3. How many buckets are there?**
- 4. Explain how textual attribute “title” is put into histogram.**
- 5. List down the bucket boundaries.**
- 6. Show the histogram as a bar-plot with value-boundaries in the X-axis and frequency in the Y-axis, as shown in the slides.**
- 7. What is the max selectivity error for this histogram? Explain/show calculation.**
- 8. How selectivity estimate is done for range query on text column such as “title”?**
- 9. What is the estimated and actual selectivities of the queries:**

select * from title where title.title < 'Race';

select * from title where title.id < '350000';

Q1. Find out the histograms for columns “id” and “title” of the “title” table

1. so the sql query is asked here which we run in postgres:

So we can get the bound of the histogram which we can see and analyse this is just fetched from the pg_stats which is made when we analysed the table .

```

SELECT histogram_bounds FROM pg_stats WHERE tablename = 'title' AND attname = 'id';
SELECT histogram_bounds FROM pg_stats WHERE tablename = 'title' AND attname = 'title';

```

2. equi depth

3.when i did \d pg_stats i saw histogram bound has a datatype of anyarray

And in class we learnt that no of bucket=no of boundary-1 (its obv like if 3 boundary 2 bucket)

But instead of manually counting it we can use a sql query:

```
job=# SELECT array_length(histogram_bounds, 1) - 1 AS number_of_buckets
FROM pg_stats
WHERE tablename = 'title' AND attname = 'id';
 number_of_buckets
-----
                100
(1 row)
```

4.so the idea for histogram bucket is that we keep the data divided like if it is age we get something like 0-10,10-20... but when i analysed to see the sql query i saw the ans as ->

{:Presentes,(#1.1447),(#1.259),(#1.4603),(#1.7268),(#12.8),(#2.37),(#3.36),(#5.173),(#8.2),(1971-08-23),(1988-09-19),(1996-01-17),(1999-03-12),(2001-02-04),(2002-10-25),(2004-07-06),(2005-09-24),(2006-12-30),(2008-04-22),(2010-02-04),(2011-05-25),(2012-06-08),(2013-05-13),"A Bride in the Morning","A relixi3n","Adversidad","All-Stars Tournament: Part 4","Animals: Moles and Voles",Attack,"Band Foto with the Throwdowns".....

From here we can see that it simply just used a technique of lexicographical ordering like dictionary , therefore the sorted value are but in buckets by string comparison like a<b <c and ...

5.The bucket boundaries are just the output of these queries liked 5 , 25115 ,....

```
job=# SELECT histogram_bounds FROM pg_stats WHERE tablename = 'title' AND attname = 'id';
```

```
SELECT histogram_bounds FROM pg_stats WHERE tablename = 'title' AND attname = 'title';
```

histogram_bounds

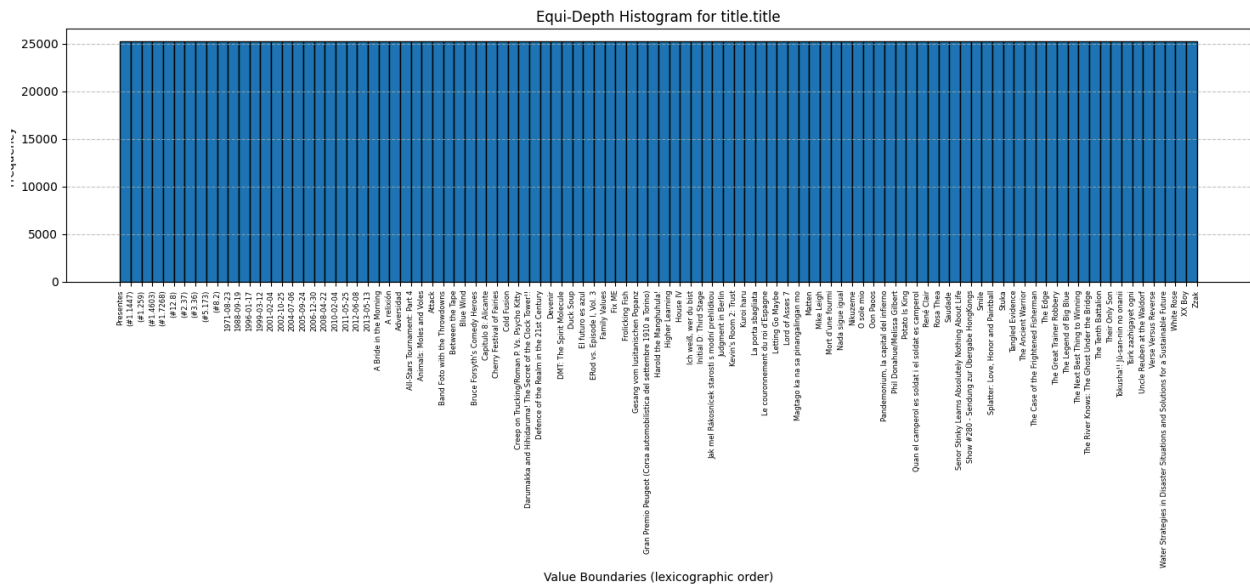
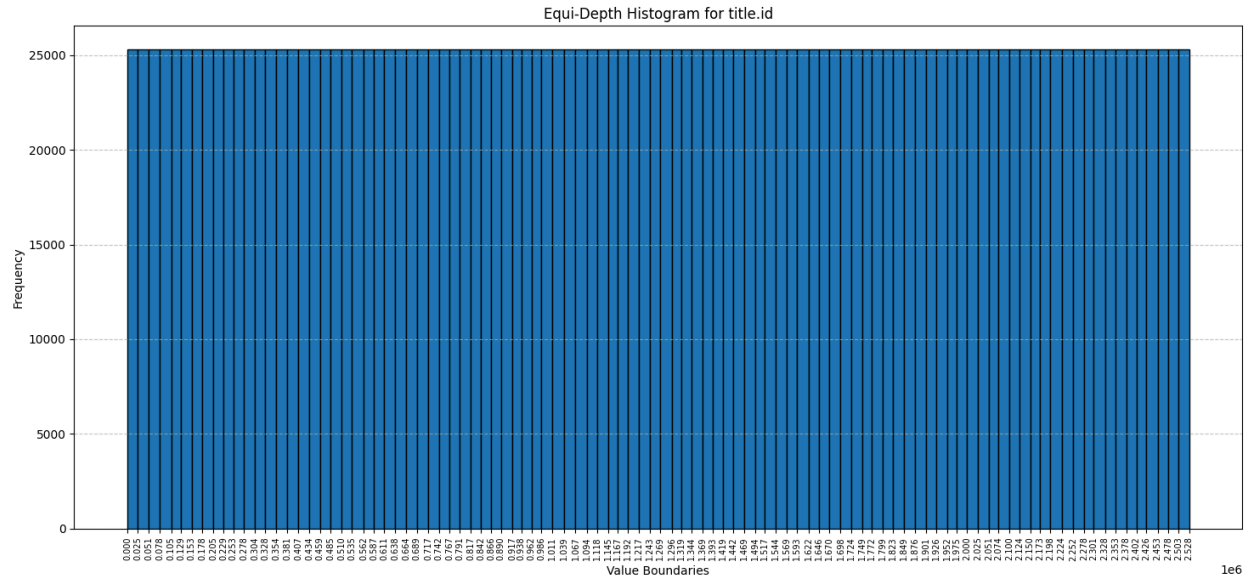
```
{5,25115,51032,77854,104855,128544,153252,178094,204585,228696,252756,277787,303649,328202,354017,380635,407446,433631,458753,484592,509939,535092,562500,587095,611308,638200,664487,689484,716642,742068,766652,791299,817195,841652,865990,889561,916584,938211,962496,985895,1011069,1039483,1067227,1093856,1118487,1145174,1167334,1191936,1216954,1243317,1269294,1295848,1318631,1343675,1369352,1393108,1419093,1442295,1468720,1494199,1517077,1543511,1569356,1593191,1622323,1646279,1670339,1698260,1724458,1749374,1772473,1798903,1822509,1848852,1875716,1901087,1926138,1951650,1974528,1999624,2024573,2051030,2074075,2099721,2124168,2149618,2173299,2197911,2224452,2251996,2278040,2300968,2327759,2353484,2378200,2402275,2425644,2453035,2478393,2503117,2528121}
```

(1 row)

histogram_bounds

{:Presentes,(#1.1447),(#1.259),(#1.4603),(#1.7268),(#12.8),(#2.37),(#3.36),(#5.173),(#8.2),(1971-08-23),(1988-09-19),(1996-01-17),(1999-03-12),(2001-02-04),(2002-10-25),(2004-07-06),(2005-09-24),(2006-12-30),(2008-04-22),(2010-02-04),(2011-05-25),(2012-06-08),(2013-05-13),"A Bride in the Morning","A relixi3n","Adversidad","All-Stars Tournament: Part 4","Animals: Moles and Voles","Attack","Band Foto with the Throwdowns","Between the Tape","Blue Wind","Bruce Forsyth's Comedy Heroes","Capitulo 8: Alicante","Cherry Festival of Fairies","Cold Fusion","Creep on Trucking/Roman P. Vs. Psycho Kitty","Darumakka and Hihidaruma! The Secret of the Clock Tower!!","Defence of the Realm in the 21st Century","Devenir","DMT: The Spirit Molecule","Duck Soup","El futuro es azul","ERod vs. Episode I, Vol. 3","Family Values","Fix ME","Frolicking Fish","Gesang vom lusitanischen Popanz","Gran Premio Peugeot (Corsa automobilistica del settembre 1910 a Torino)","Harold the Manghuhula!","Higher Learning","House IV","Ich wei, wer du bist","Initial D: Third Stage","Jak mel Rkosncek starosti s modn prehldkou","Judgment in Berlin","Kevin's Room 2: Trust","Kuroi haru","La porta sbagliata","Le couronnement du roi d'Espagne","Letting Go Maybe","Lord of Asses 7","Magtago ka na sa pinangalingan mo","Matten","Mike Leigh","Mort d'une fourmi","Nada sigue igual","Nikuzeme","O sole mio","Oon Paoos","Pandemonium, la capital del infierno","Phil Donahue/Melissa Gilbert","Potato Is King","Quan el camperol es soldat i el soldat es camperol","Ren Clair","Rosa Thea","Saudade","Senor Stinky Learns Absolutely Nothing About Life","Show #280 - Sendung zur bergabe HongKongs","Smile","Splatter: Love, Honor and Paintball","Stuka","Tangled Evidence","The Ancient Warrior","The Case of the Frightened Fisherman","The Edge","The Great Trainer Robbery","The Legend of Big Blue","The Next Best Thing to Winning","The River Knows: The Ghost Under the Bridge","The Tenth Battalion","Their Only Son","Tokusha!! J-san-nin no onanii","Tsirk zazhigayet ogni","Uncle Reuben at the Waldorf","Verse Versus Reverse","Water Strategies in Disaster Situations and Solutions for a Sustainable Future","White Rose","XX Boy","Zzak

6.The code for plots will be in plot1.py and plot2.py The plots are ->



7.so since it is an equi depth histogram , the error is 1/ no of buckets

Therefore error is 1/100 -> 1%

To prove this ->if each bucket has aprox 25,283 tuples and the total no of rows is approx 100*25283.

Therefore max error can be of one bucket ->25,283/100*25,283=1/100=1%

8. For a range query on title column which is a text column, as we saw that postgres uses lexicographical ordering to find buckets and divide the boundaries so to find if $\text{title} < 'xyz'$, .All the buckets which are before it fully satisfy the condition so we add them and then postgres uses a genius idea that it converts that bucket which is to be processes from string to number and then uses a simple concept of linear interpolation to estimate the ans, then final ans is all previous bucket + estimated fraction by linear interpolation = selectivity.

9.

These are the formulas for the selectivity

Actual Selectivity = (Actual Rows Returned by Query) / (Total Rows in Table)

Estimated Selectivity = (Estimated Rows) / (Total Rows in Table)

Therefore i will run some queries :

(i)

actual = $1919462/2528312=0.7592 = 75.92\%$

estimated= $1940408/2528312=0.76744 = 76.74\%$

```
job=# SELECT count(*) FROM title;
      count
-----
 2528312
(1 row)

job=# EXPLAIN SELECT *
FROM title
WHERE title < 'Race';
              QUERY PLAN
-----
Seq Scan on title  (cost=0.00..67611.90 rows=1940408 width=94)
  Filter: ((title)::text < 'Race'::text)
(2 rows)

job=# SELECT count(*)
FROM title
WHERE title < 'Race';
      count
-----
 1919462
(1 row)

job=#
```

(ii)

```
job=# EXPLAIN SELECT *  
FROM title  
WHERE id < 350000;
```

QUERY PLAN

```
-----  
Index Scan using title_pkey on title (cost=0.43..16628.62 rows=350028 width=94)  
  Index Cond: (id < 350000)  
(2 rows)
```

```
job=# SELECT count(*)  
FROM title  
WHERE id < 350000;  
count
```

```
-----  
349999  
(1 row)
```

Actual =349999/2528312 =0.13844 = 13.844%

estimated=350028/2528312 = 0.13843 =13.843%

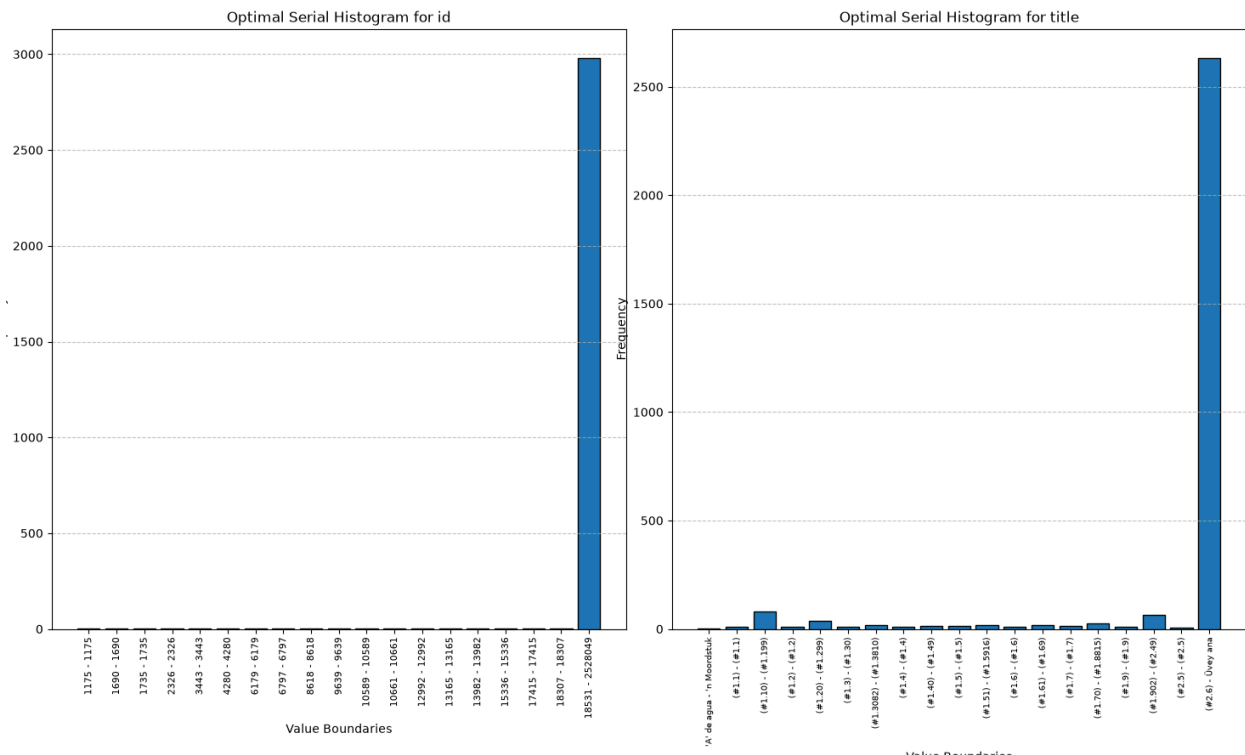
Q2.

Q2. Generate a code to create and visualize optimal serial histogram of 10-50 buckets (actual number is as per your choice) from column data fetched from the database. Now, use that code to create histograms for “id” and “title” columns of 3000 random samples drawn from the “title” table (use TABLESAMPLE command for sampling the table).

- 1. Show the histograms as a bar-plot with value-boundaries in the Y-axis and frequency in the X-axis, as shown in the slides.**
- 2. How much time did it take to build?**
- 3. List down the frequency bucket boundaries.**
- 4. What is the max selectivity error for this histogram? Explain/show calculation.**
- 5. Repeat the sampling for 1000 and 5000, and build the corresponding histograms using the same code. Record the timings.**
- 6. Using the time-data for 1000, 3000 and 5000 samples, extrapolate the time required to build histogram for the full table (you may use linear regression). List down the extrapolated times.**
- 7. Create one plot to show the sample size vs time relationship.**
- 8. Create one plot to show the sample size vs max selectivity error relationship.**
- 9. If you choose less number of buckets, what will happen? What will happen for more number of buckets?**

Ans -> **1.** For that i have a code named histogram.py to build this

The plot for title and id i have combined in a single plot so one doesnt need to change the value in the field of target column



2. The time for building is measured for taking 3000 random values and making them go in histogram which includes making bucket , bucket freq calc , DP , sorting . and we find this time by using time.time() fucn in python of time library .

OPTIMAL SERIAL HISTOGRAM FOR ID

Samples: 3000

Buckets: 19

Minimum SSE: 0.0

Build Time: 11.030512 seconds

Maximum Selectivity Error: 0.0000000

Maximum Selectivity Error (%): 0.000000%

```
=====
OPTIMAL SERIAL HISTOGRAM FOR TITLE
=====
Samples: 3000
Buckets: 19
Minimum SSE: 116.65767134912744
Build Time: 9.906562 seconds
Maximum Selectivity Error: 0.00094253
Maximum Selectivity Error (%): 0.094253%
```

3.the freq bucket boundaries are

0-500 , 500-1000 , 1000-1500 , 1500-2000 , 2000-2500 , 2500-3000

And if we are talking about the data point boundary machine is making then

```
Frequency Bucket Boundaries:
Bucket 1: 898 - 898 Frequency = 1
Bucket 2: 1040 - 1040 Frequency = 1
Bucket 3: 2088 - 2088 Frequency = 1
Bucket 4: 2500 - 2500 Frequency = 1
Bucket 5: 4657 - 4657 Frequency = 1
Bucket 6: 8324 - 8324 Frequency = 1
Bucket 7: 8715 - 8715 Frequency = 1
Bucket 8: 9096 - 9096 Frequency = 1
Bucket 9: 9197 - 9197 Frequency = 1
Bucket 10: 9630 - 9630 Frequency = 1
Bucket 11: 10112 - 10112 Frequency = 1
Bucket 12: 10816 - 10816 Frequency = 1
Bucket 13: 12364 - 12364 Frequency = 1
Bucket 14: 12797 - 12797 Frequency = 1
Bucket 15: 13251 - 13251 Frequency = 1
Bucket 16: 13827 - 13827 Frequency = 1
Bucket 17: 13838 - 13838 Frequency = 1
Bucket 18: 14122 - 14122 Frequency = 1
Bucket 19: 14310 - 2528202 Frequency = 2982
```

```
Frequency Bucket Boundaries:
Bucket 1: 'Episode 3056' - (#1.0) Frequency = 2
Bucket 2: (#1.1) - (#1.1) Frequency = 12
Bucket 3: (#1.10) - (#1.188) Frequency = 69
Bucket 4: (#1.19) - (#1.19) Frequency = 7
Bucket 5: (#1.192) - (#1.1993) Frequency = 2
Bucket 6: (#1.2) - (#1.2) Frequency = 14
Bucket 7: (#1.20) - (#1.29) Frequency = 32
Bucket 8: (#1.3) - (#1.3) Frequency = 9
Bucket 9: (#1.30) - (#1.3845) Frequency = 20
Bucket 10: (#1.4) - (#1.4) Frequency = 12
Bucket 11: (#1.40) - (#1.5982) Frequency = 29
Bucket 12: (#1.6) - (#1.6) Frequency = 8
Bucket 13: (#1.60) - (#1.694) Frequency = 11
Bucket 14: (#1.7) - (#1.7) Frequency = 14
Bucket 15: (#1.70) - (#1.79) Frequency = 7
Bucket 16: (#1.8) - (#1.8) Frequency = 6
Bucket 17: (#1.80) - (#2.43) Frequency = 88
Bucket 18: (#2.5) - (#2.6) Frequency = 10
Bucket 19: (#2.67) - È caduta una donna Frequency = 2648
```

4.

$$\text{Max Selectivity Error} = \max_b \left(\max_{i \in b} \frac{|f_i - \bar{f}_b|}{N} \right)$$

- b represents a specific bucket.
- i represents a distinct value within bucket b .
- f_i is the actual frequency of value i .
- $\bar{f}_b$ is the mathematical mean frequency of bucket b (calculated as the total number of rows in the bucket divided by the number of distinct values in the bucket).
- N is the total cardinality (total rows) of the table.

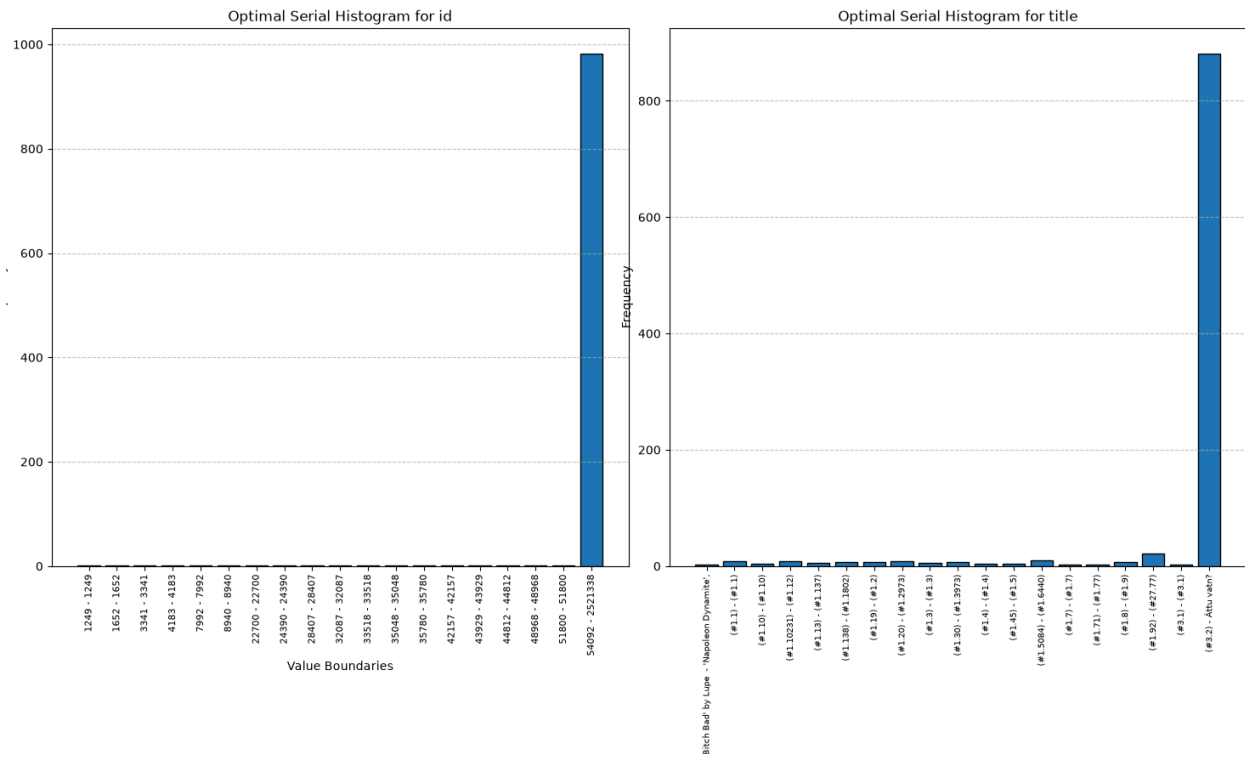
So if i use this formula and i think of automating it in my code by writing a python code which automatically does it for me ,

The python script is somewhat like

We have a i/p of bucket boundary , freq of each distinct value and total no of sampled row=0 (N) and for each bucket b i will take total freq in that bucket and distinct count in bucket and the mean freq is total freq/distinct count now for each distinct value i in bucket b i will do error =mod(freq[i]-mean_freq)/N and if error >ax_error i change that max_error and then return that .

By this the value i get for error in id is 0 and that is absolutely fine because the id is the pk and all will have diff so no repetition , the actual freq and estimated avg are identical so no estimation error, therefore its ok and for title one error is 0.0942% so this small error indicates that the optimal histogram provides a close approximation of actual distribution .

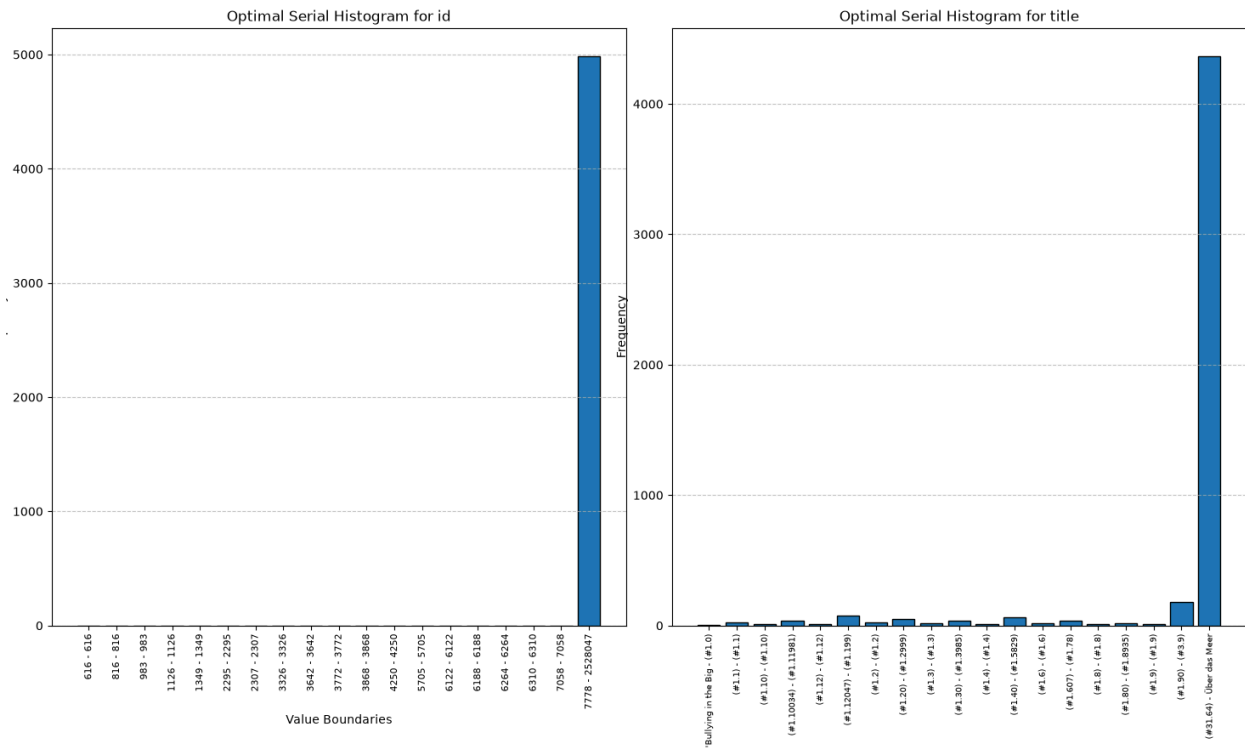
5. This one for 1000 samples



The build time for 1000 is for id ->1.226 second and title is ->1.116 sec

This one for 5000 samples->

The build time for 5000 is for id ->31.3998 sec and title is ->27.278 sec

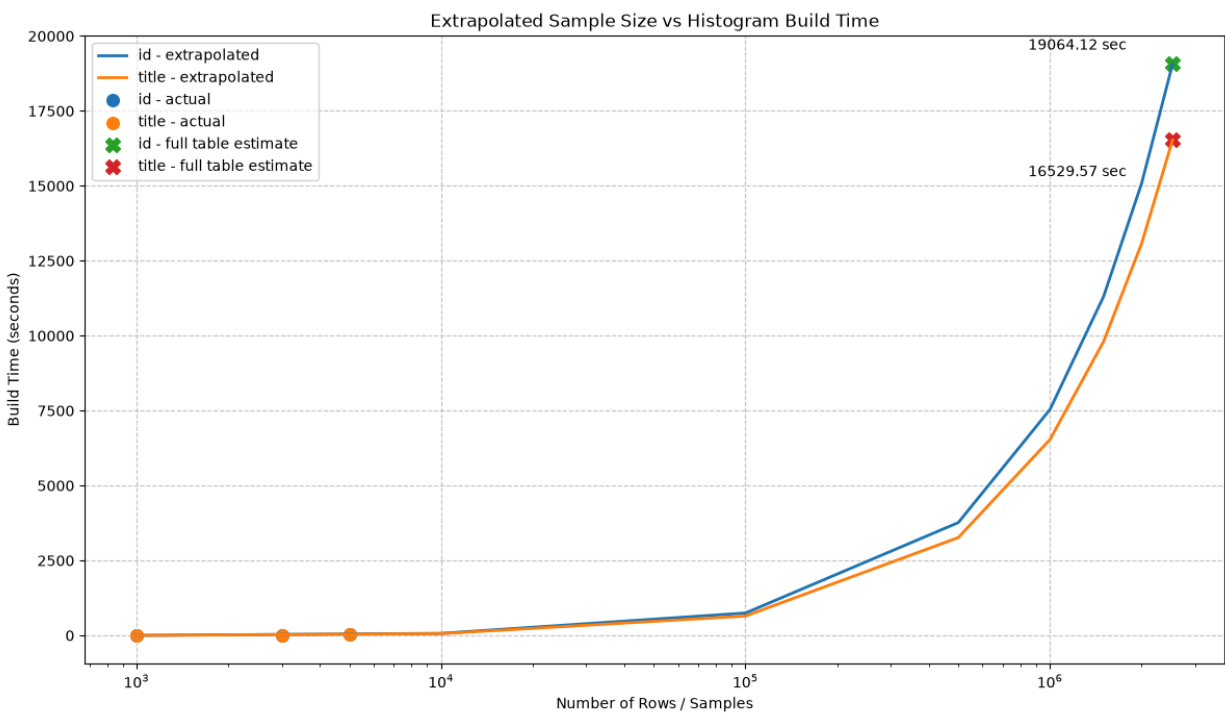
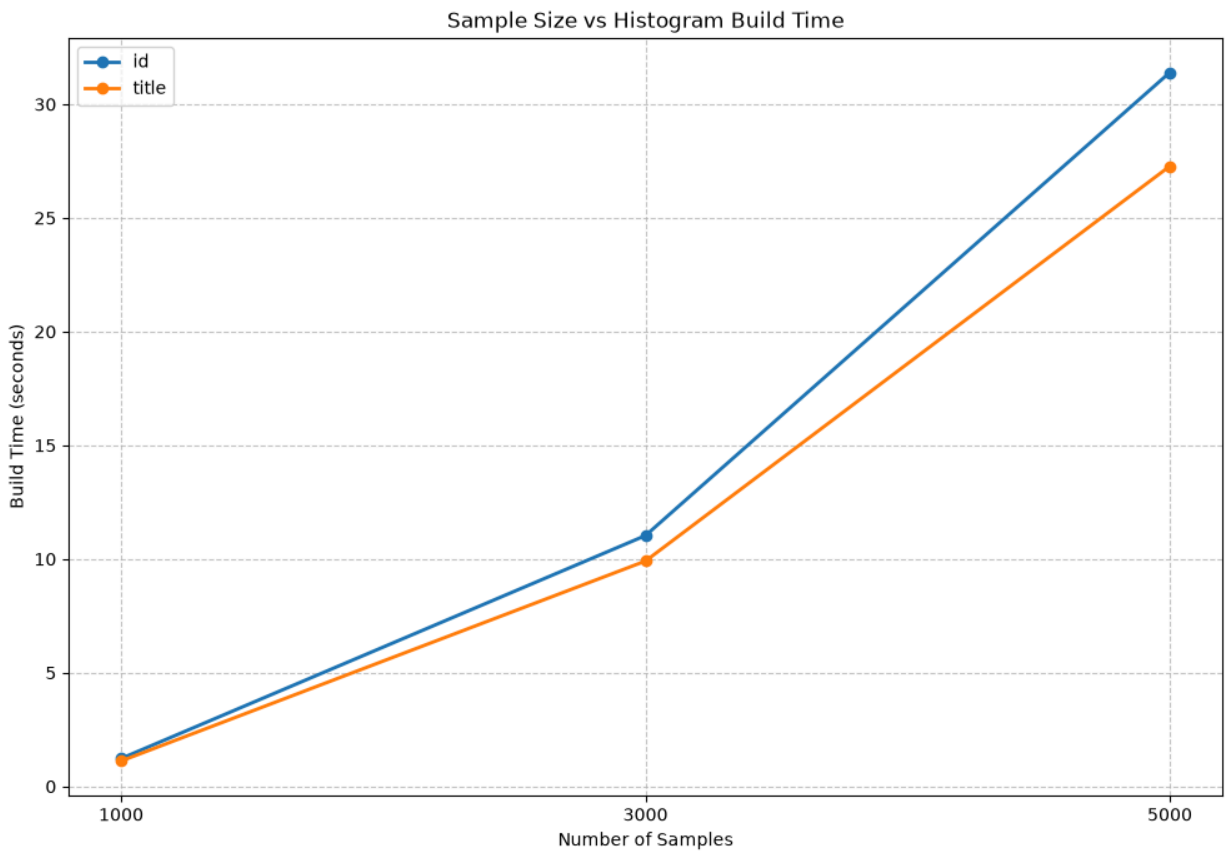


6. For this I am using scikit learn library as it has linear regression inbuilt in this (code is in histogram.py)

```
=====
EXTRAPOLATION FOR 'ID' COLUMN
=====
Equation: Time = (0.007543 * rows) + -8.078250
Extrapolated Time for 2,528,312 rows:
Seconds : 19,064.12
Hours   : 5.30
Days    : 0.22

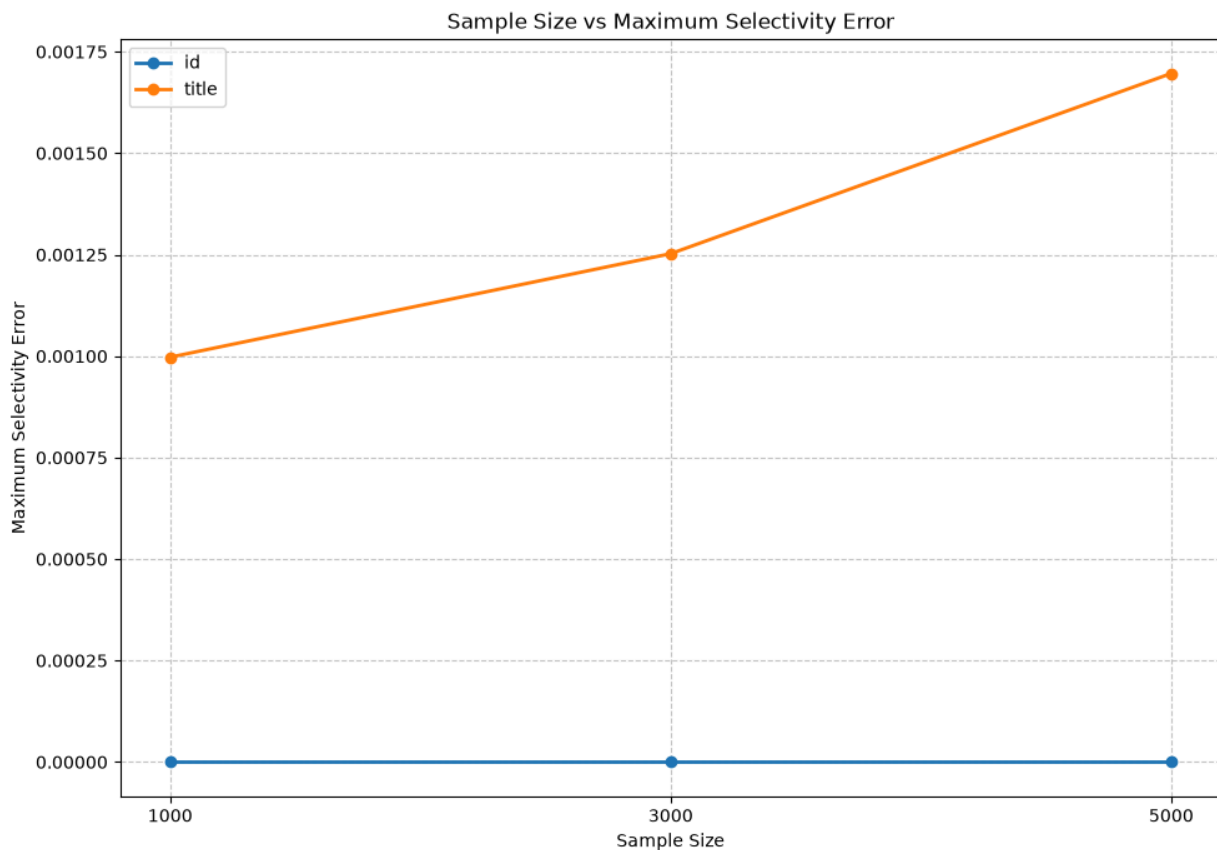
=====
EXTRAPOLATION FOR 'TITLE' COLUMN
=====
Equation: Time = (0.006541 * rows) + -6.853500
Extrapolated Time for 2,528,312 rows:
Seconds : 16,529.57
Hours   : 4.59
Days    : 0.19
```

7.The code for plot is in histogram.py



8. The script to find the max selectivity is in selectivity.py and the brief idea about what I am doing there is that my code builds optimal histogram for diff samples by dp, for each bucket it calculates the freq and then we find the avg and then the max selectivity error which is just the $\text{abs}(\text{bucket avg} - \text{actual freq}) / \text{total sample}$. then we finally plot btw 1000, 3000, 5000 and corresponding error, it helps us see how accuracy changes.

```
(venv) ayushpatel@Ayushs-MacBook-Air Downloads % python3 selectivity.py
Sample Size = 1000
ID Max Selectivity Error = 0.00000000 (0.0000%)
TITLE Max Selectivity Error = 0.00099739 (0.0997%)
-----
Sample Size = 3000
ID Max Selectivity Error = 0.00000000 (0.0000%)
TITLE Max Selectivity Error = 0.00125238 (0.1252%)
-----
Sample Size = 5000
ID Max Selectivity Error = 0.00000000 (0.0000%)
TITLE Max Selectivity Error = 0.00169647 (0.1696%)
-----
```



9. This is a pretty straightforward question that if we select less bucket then each bucket will cover more range of values and we need less computation but the avg freq will be more deviated than actual data as it is representing a large set of values here, so selectivity error

generally increases , and if i choose more buckets i will have more overhead and each bucket has less values , so the individual freq are more accurate and selectivity error is less , but computation is heavy , therefore few bucket -> simpler histogram but higher error and more buckey -> more accurate histogram but higher cost , so we try to build a balance of both in real life .